МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Информационные системы

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Информатика и информационные технологии»
Тема: Простой интерпретатор арифметических действий

Студент гр. 6374 _____ Акимов В.А.

Преподаватель _____ Дюков Н.В.

Санкт-Петербург
2026

Цель работы.

Написать код на язык Python, представляющий собой интерпретатор арифметических выражений, который принимает на вход строку с выражением, поддерживает << +, -, *, /, (,) >> и поддерживает целые дробные числа с функцией sin() и exp().

Основные теоретические положения.

Интерпретатор - это программа, которая выполняет код, написанный на языке программирования, построчно, анализируя и исполняя каждую инструкцию непосредственно в момент её прочтения. В отличие от компилятора, который переводит весь исходный код в машинный или байт-код целиком до запуска программы, интерпретатор работает «на лету», обрабатывая код в процессе выполнения.

Этапы работы интерпретатора:

1.Лексический анализ. Интерпретатор разбивает исходный код на токены — минимальные значимые единицы: ключевые слова, идентификаторы, литералы (числа, строки), операторы, скобки.

2.Синтаксический анализ. На основе токенов строится абстрактное синтаксическое дерево — древовидная структура, отражающая структуру программы. Каждый узел дерева представляет конструкцию языка (например, операцию сложения или вызов функции). Если на этом этапе обнаруживается синтаксическая ошибка (неправильная последовательность токенов), интерпретатор прекращает работу и выводит сообщение об ошибке.

3.Семантический анализ. На этом этапе проверяется смысловая корректность кода: проверяются типы переменных, область видимости, совместимость операций.

4.Создание промежуточного представления кода. Для простого интерпретатора этот пункт не обязателен. Но все зависит от типа интерпретатора.

5.Исполнение. Вычисление результата.

Разработка программы.

Общая структура. Код построен на рекурсивном спуске — это классический способ разбора выражений. Главная функция calculate управляет процессом, а подфункции разбирают разные части выражения в соответствии с приоритетами операций.

Описание функций:

1.Calculate(expression) это главная функция, которая принимает строку expression и возвращает результат вычисления.

2.Skip_spaces() пропускает пробелы в строке. Использует nonlocal pos, так как изменяет внешнюю переменную pos.

3.Read_number() читает число из строки. Также использует nonlocal pos. Проверяет, что за цифрой (или точкой для дробей) действительно следует цифра или точка. Если число не найдено, возбуждает исключение ValueError.

4.Plus_minus() обрабатывает сложение и вычитание (операции низкого приоритета). В цикле проверяет наличие знаков + или -.

5.Multiply_divide() работает с умножением и делением (операции высокого приоритета). Вызывает для получения первого множителя функцию unary(). В цикле проверяет наличие * или /. При делении добавляет проверку на деление на ноль.

6.Unary() обрабатывает унарные операции — знаки перед числом или скобкой. Если встречается знак -, функция рекурсивно вызывает себя для части выражения после знака и возвращает противоположное значение.

7.Element_spec() обрабатывает элементы, включая функции и скобки.

8. Function_value_spec(function) вычисляет значение функции (например, sin или exp). Проверяет наличие открывающей скобки, затем вызывает plus_minus() для аргумента, проверяет закрытие скобки и применяет функцию к полученному числу.

Основные идеи кода:

1.Разделение на уровни приоритета. То есть работает на примере синтаксического дерева.

2.Использование pos для парсинга. Эта переменная какой символ на данный момент обрабатывается. Это порзволяет последовательно проходить по выражению, не теряя контекст

3.Обработка пробелов. Делает ввод выражения для пользователя более удобнее.

4.Рекурсивный разбор для обработки скобок и функций. Проверяет открытие и закрытие скобок и вычисление специальных функций.

5.Обработка унарных операций. Если перед числом или после него встречается ‘-’ функция рекурсивно вычисляет то, что следует.

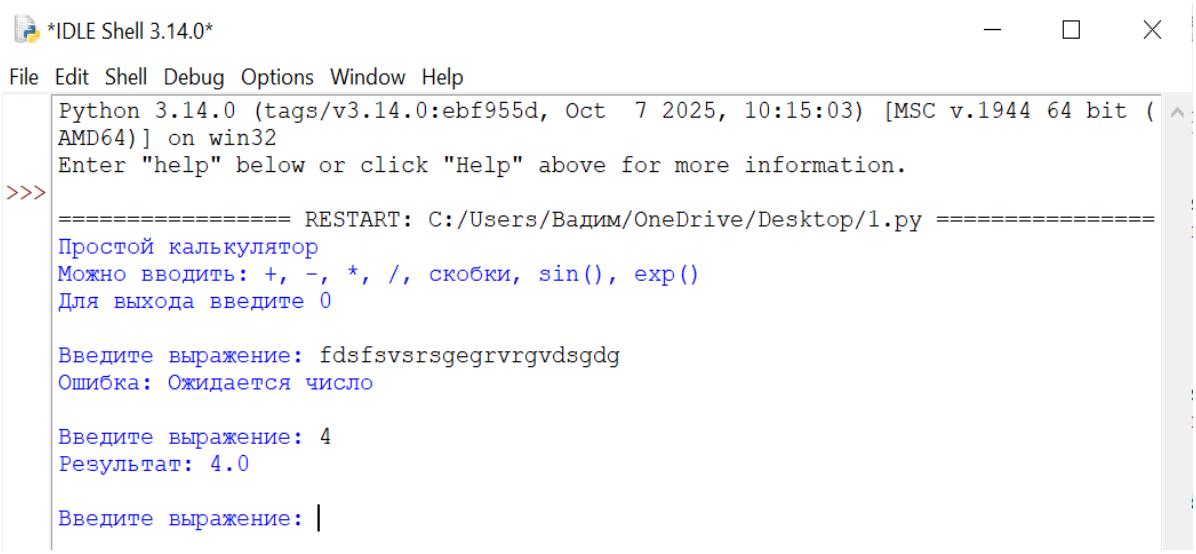
6.Обработка математических ошибок.

7.Программа постоянно запрашивает код пока не будет введен ‘0’.

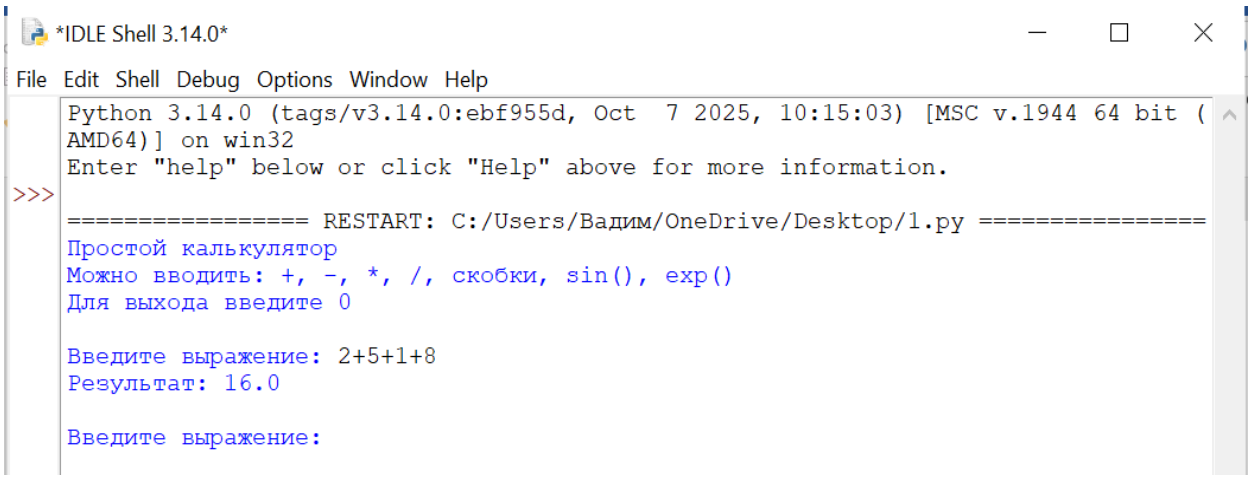
Результаты работы программы.

1.Проверка ввода:

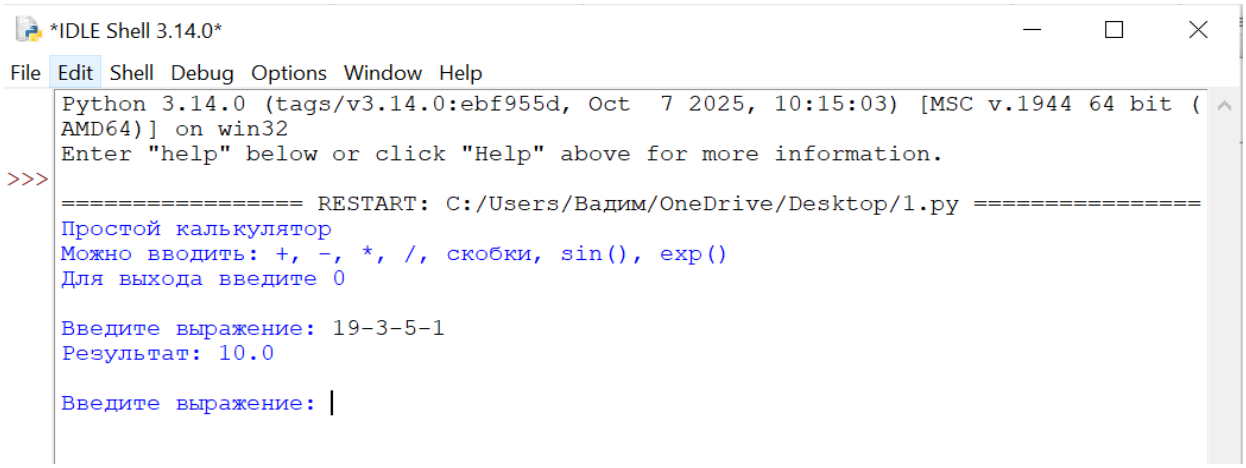
Проверка сначала некорректного ввода, а потом корректного.



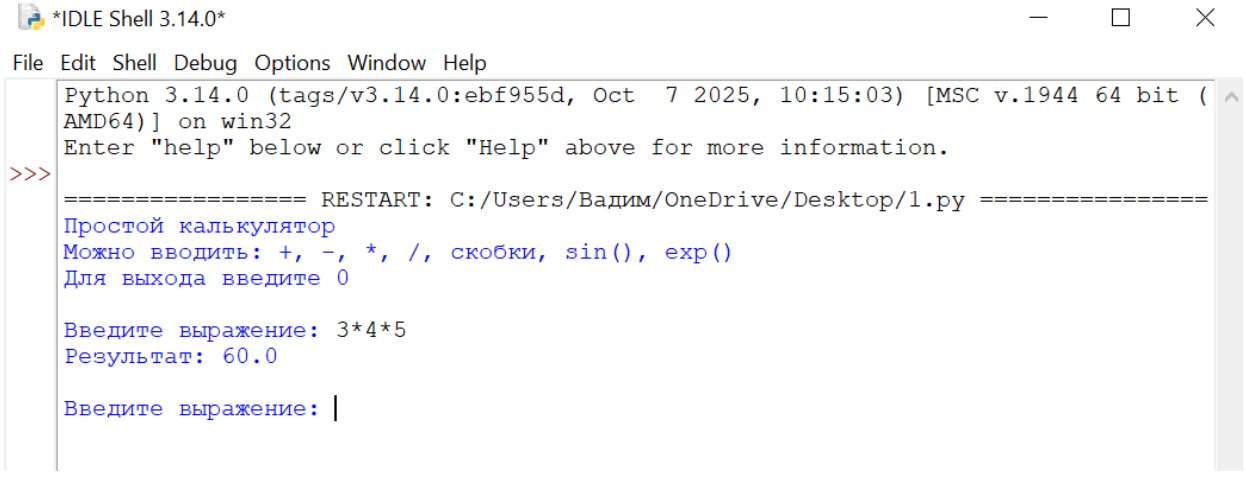
2.Проверка суммы:



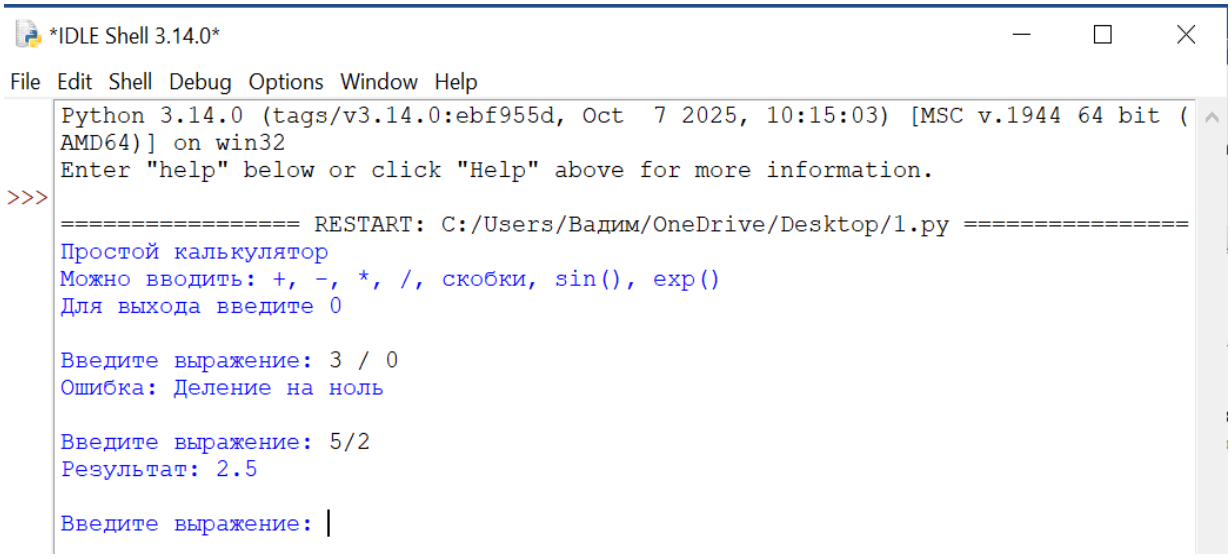
3.Проверка разности:



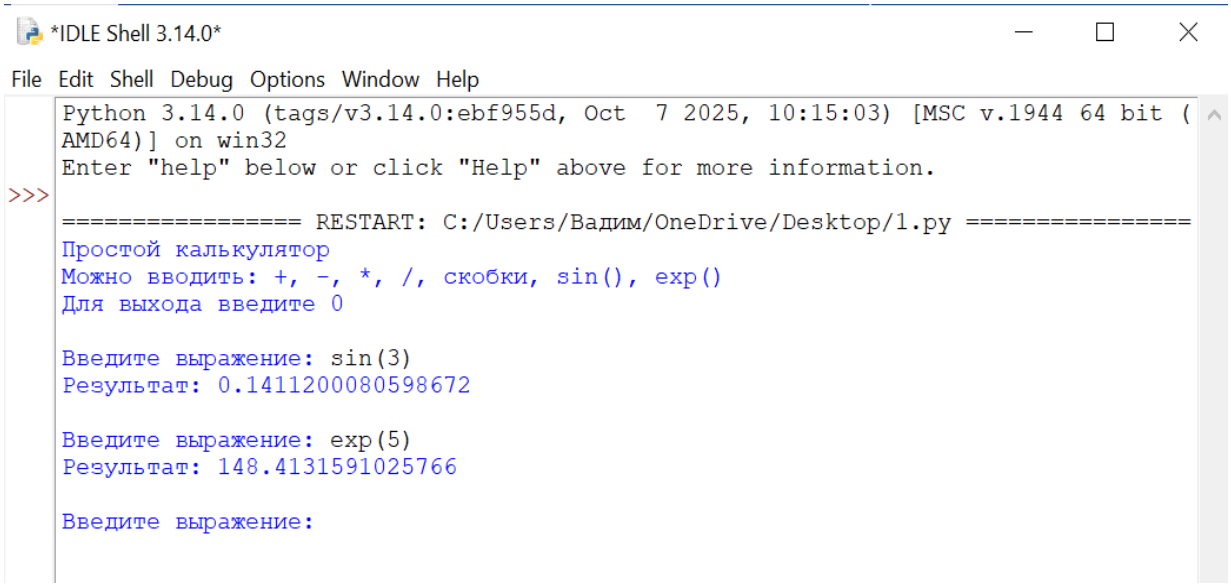
4.Проверка умножения:



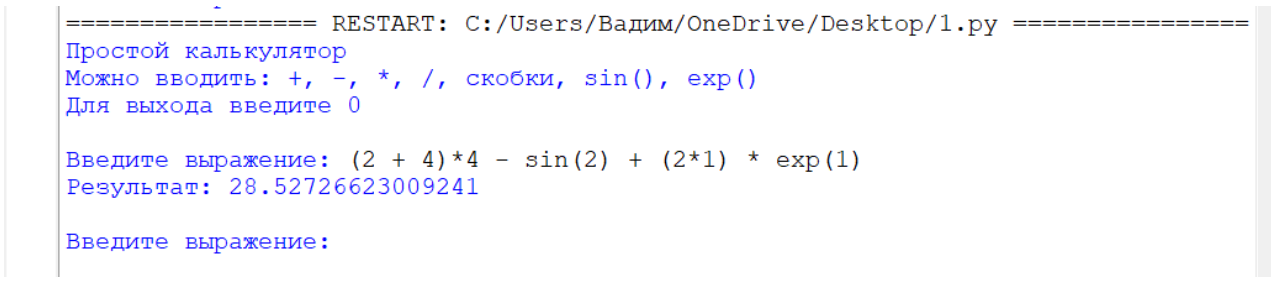
5.Проверка деление на ноль и без него.



6.Проверка синуса и экспоненты.



7.Проверка обработки результатов по действиям.



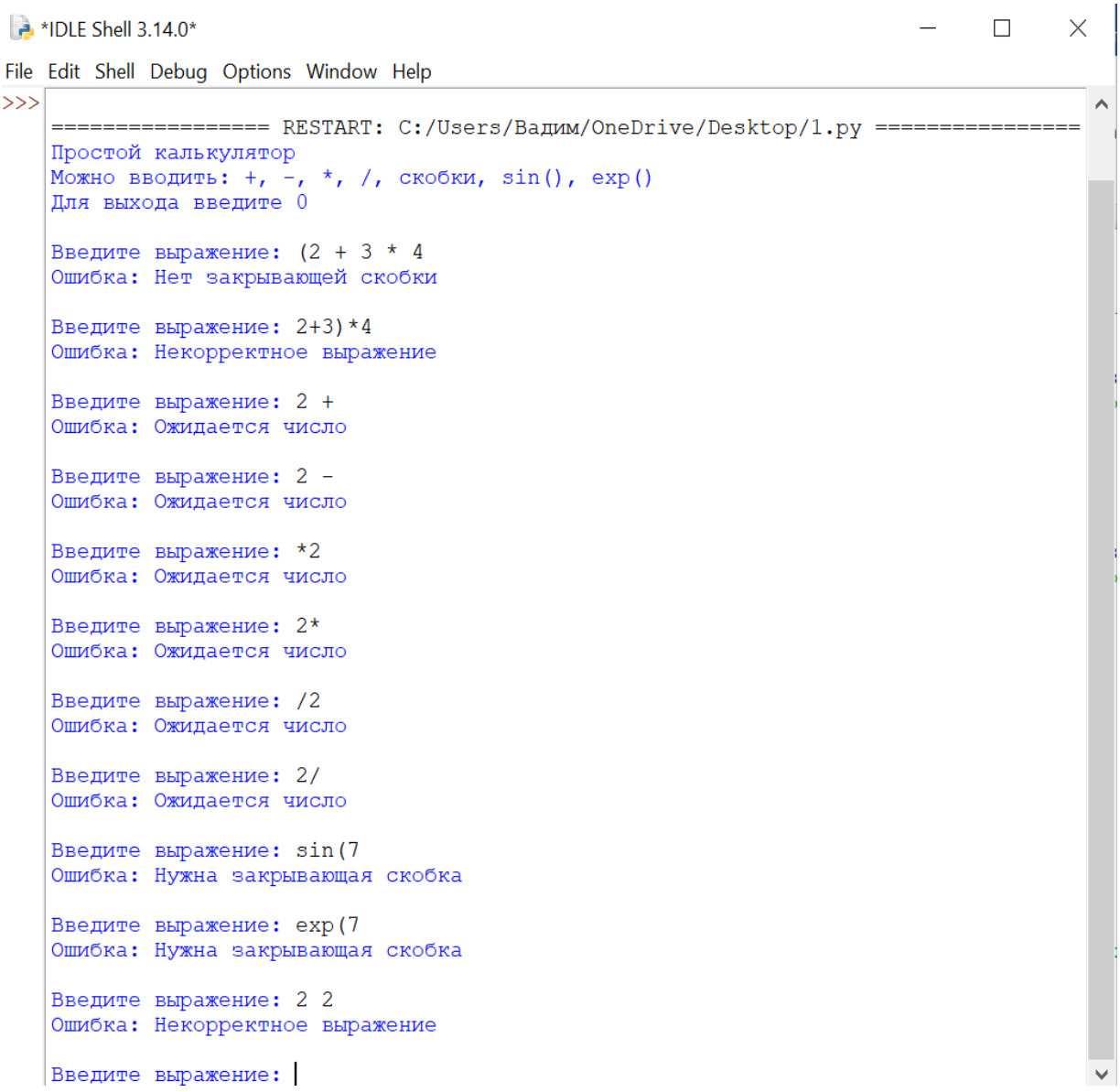
8.Проверка унарного минуса.

```
================ RESTART: C:/Users/Вадим/OneDrive/Desktop/1.py ================
Простой калькулятор
Можно вводить: +, -, *, /, скобки, sin(), exp()
Для выхода введите 0

Введите выражение: -(4+8+1)
Результат: -13.0

Введите выражение:
```

9.Проверка некорректного ввода математического действия.



Выводы.

Разработан простой интерпретатор арифметических действий, написанный на языке программирования Python. Реализовано разделение на уровни приоритета, лексический анализ, синтаксический анализ и подсчет результата. Программа поддерживает целые и вещественные числа, +, -, *, / , синус, экспоненту, унарный минус, скобки без ошибок. Программа выдает ожидаемый результат(проверка осуществлялось по аппаратному калькулятору).

Цель работы достигнута.

ПРИЛОЖЕНИЕ